

ModelMatch

Choosing the right AI model in seconds, not hours — explainer for the team

The 20-second version (say this if you forget everything else)

There are over a million AI models online and nobody can tell which one fits their task, their language and their laptop. **ModelMatch** lets you type what you want to do and pick your device. It then finds the best-fitting models, **runs them on your own sample text**, and shows the results side by side, with Hindi, Kannada and Tamil support built in.

1. Problem statement

Developers, students and startups cannot easily find an AI model that fits their task, their language and their hardware.

Hugging Face alone hosts over a million models. Its search sorts by popularity (downloads and likes), which does not tell you:

- whether the model works for **your specific job**
- whether it will **fit in your GPU or laptop memory**
- how good it is in **Indian languages** like Hindi, Kannada or Tamil
- whether its **licence** allows commercial use

Result: people download several models and test them by hand. That wastes hours, data and money.

2. What we are solving

Problem today	How ModelMatch fixes it
Wrong model (ranked by popularity)	Understands your task and ranks models by fit
Hardware limits (will it even run?)	Calculates which models fit your device and the best precision to use
No proof before downloading	Live test : runs top models on your own input
Indian-language gap	Indian-language filter (Hindi, Kannada, Tamil)

3. The solution we are going to build

A web app where the user types a plain-English task, picks a device, and gets a tested, ranked shortlist.

How it works, step by step:

Step	What happens
1. Input	User describes the task (e.g. ' <i>summarise Kannada news</i> ') and picks a device (e.g. 8 GB GPU).
2. Parse	An LLM turns the sentence into needs: task type, language, memory limit.
3. Fetch	The Hugging Face Hub API pulls candidate models for that task.
4. Fit engine	Calculates memory per model and picks the best precision that fits. Ranks the results.
5. Live test	Runs the top 2–3 models on the user's sample text, outputs shown side by side.
Output	Memory needed, best quantization, licence, and a real sample result for each model.

The one formula to remember

Memory needed = parameters x bytes per parameter x 1.2

The 1.2 is a 20% buffer for overhead. A 7B-parameter model needs about 16.8 GB at 16-bit and 8.4 GB at 8-bit, so it will **not** fit an 8 GB GPU. At 4-bit it needs about 4.2 GB, so it **fits**. Shrinking the precision like this is called **quantization**, and our app picks the best precision that fits automatically.

Precision	Memory (7B model)	Fits 8 GB GPU?
16-bit	16.8 GB	No
8-bit	8.4 GB	No
4-bit	4.2 GB	Yes

4. Why we can win

Most teams will show a list of models. We show the models **actually running on the judge's own input**. Our build time goes into three things:

- **Live test:** top 2–3 models run on the sample text and outputs appear together.
- **Hardware fit:** shows which models fit in 8 GB and which precision to use.
- **Indian-language filter:** Hindi, Kannada, Tamil.

Safety net: prepare **3 pre-tested queries** so the demo never fails on stage.

5. Why nothing like it exists

Only tick a box in the table if you have actually built that feature.

Feature	HF search	Leaderboards	ModelMatch
Plain-English input	No	No	Yes
Hardware filter	No	Partly	Yes
Tests on your input	No	No	Yes
Indian-language focus	No	Rarely	Yes

6. Impact and business idea

- **Impact:** hours become seconds; smaller GPUs and lower cost; a clear starting point for beginners.
- **Business:** free for students, paid API for teams, vendor-sponsored benchmarks.

7. Roadmap

When	What
Now	Working picker, live test, Indian-language filter
Next 3 months	Measured speed on real devices, licence checks
Later	Public API, more languages

8. Slide-by-slide: what to say

Slide	Talking point
1 Title	ModelMatch: choosing the right AI model in seconds, not hours.
2 Problem	A million models, ranked by popularity not fit. Memory needs are unclear before download. Indian-language quality is hard to compare.
3 Question	Can a task and a device produce a shortlist developers trust? Our answer: yes, and we prove it by testing.
4 What we solve	Wrong model, hardware limits, no proof, language gap. One fix for each.
5 Approach	Describe task and hardware, LLM parses needs, Hub API fetches candidates, fit engine ranks and tests.
6 Quantization	A 7B model needs 16.8 GB at 16-bit but only 4.2 GB at 4-bit, so it fits an 8 GB GPU.

Slide	Talking point
7 Demo	Show the app: plain-language task, hardware-aware ranking, try each model on your sample.
8 Comparison	No existing tool combines task parsing, hardware fit and live testing.
9 Impact	Faster, cheaper choices. Free for students, paid API for teams.
10 Next steps	Real device benchmarks, licence checks, public API.
11 Conclusion	Fast, fit-aware, testable. Share demo link or QR.

9. Pitch opener (say this first)

“Raise your hand if you've wasted an hour picking an AI model. Watch me do it on Hugging Face. Now watch ModelMatch do it in ten seconds.”

10. Before you present

- If the hackathon has an **official problem statement or track**, put its exact wording on the problem slide and show how ModelMatch answers it. Judges score match to the PS.
- Add one **real number** from a quick survey of 10 classmates (e.g. 'X of 10 wasted an hour picking a model').
- Fill in the placeholders: team name, hackathon, track, date, email, demo link, GitHub.
- We cannot promise a win, but a working live demo is our strongest edge.